

```
"""
gr_fixed_point.py - Find the field state  $\psi^*$  that minimizes the Einstein residual.
```

```
Runs N trajectories from different initial conditions.
Finds the one that produces the minimum  $||\langle T_{\mu\nu} \rangle - \kappa G_{\mu\nu}^{(1)} ||_F$  at closure.
Then gradient-refines from that seed to find  $\psi^*$ .
```

```
The fixed point  $\psi^*$  satisfies:
 $\langle T_{\mu\nu}[\psi^*] \rangle = \kappa \cdot G_{\mu\nu}^{(1)}[h[\psi^*]]$ 
```

```
This is a genuine GR-consistent field configuration derived from UMA dynamics.
"""
```

```
import numpy as np
import math
from dataclasses import dataclass
from typing import List, Optional, Tuple
```

```
@dataclass
class TrajectoryResult:
    seed: int
    einstein_residual_final: float
    einstein_residual_min: float
    closure_step: Optional[int]
    z_final: np.ndarray
    h_final: np.ndarray      # (2,2) metric perturbation at final state
    kappa: float
```

```
@dataclass
class GRFixedPoint:
    z_star: np.ndarray      # field state at GR fixed point
    h_star: np.ndarray      # metric at fixed point
    T_star: np.ndarray      # stress tensor at fixed point
    G_star: np.ndarray      # Einstein tensor at fixed point
    einstein_residual: float #  $||T - \kappa G|| / ||T||$  at fixed point
    kappa: float
    seed_used: int
    n_trajectories: int
```

```
class GRFixedPointFinder:
    """
```

Finds ψ^* minimizing the Einstein residual across N trajectories.

Usage::

```
finder = GRFixedPointFinder(cfg, n_trajectories=20)
fixed_point = finder.find(verbose=True)
print(f"GR fixed point: residual={fixed_point.einstein_residual:.4f}")
"""

def __init__(self, cfg, n_trajectories: int = 20,
              n_steps: int = 250, T_window: int = 30):
    self.cfg = cfg
    self.n_traj = n_trajectories
    self.n_steps = n_steps
    self.T_window = T_window
    self.N = cfg.grid.N
    self.dx = 1.0 / cfg.grid.N

    kx = 2*math.pi*np.fft.fftfreq(self.N, d=self.dx)
    KX, KY = np.meshgrid(kx, kx)
    self.k2 = KX**2 + KY**2
    self.k2[0,0] = 1.0

def _run_trajectory(self, seed: int) -> TrajectoryResult:
    from uma import UMAClient
    from uma.core.state import FieldPosterior
    from uma.semantic.friction import SemanticFriction, SemanticFrictionConfig
    from uma.semantic.constants import CALIBRATED_DH_DT_FLOOR, CALIBRATED_FIE

    cfg = self.cfg
    N = self.N
    rng = np.random.default_rng(seed)

    client = UMAClient(cfg)
    psi0 = 0.3*(rng.standard_normal((N,N))+1j*rng.standard_normal((N,N)))
    client.initialize(psi0, sigma0=0.05, dt=0.04)

    # Burn-in for z_target
    for _ in range(50):
        psi = client.projection.lift(client.posterior_state.mean)
        psi = client.dynamics.step(psi, 0.04, rng=rng)
        z = client.projection.project(psi)
        client.posterior_state = FieldPosterior.full(z, client.posterior_state)
    z_target = client.posterior_state.mean.copy()

    # Friction config
    fr_cfg = SemanticFrictionConfig(
        convergence_dz_dt=CALIBRATED_DH_DT_FLOOR,
```

```

        min_steps_before_closure=10,
        step_weight=0.025,
    )

def H_fn(z):
    p = client.projection.lift(z)
    v = client.dynamics.hamiltonian(p)
    return float(v.real if hasattr(v, 'real') else v)

friction = SemanticFriction(z_target, fr_cfg, hamiltonian_fn=H_fn)
friction.reset(client.posterior_state)

T_buffer = []
residuals = []
closure_step = None
z_final = z_target.copy()
h_final = np.zeros((2,2))
T_final = np.zeros((2,2))
G_final = np.zeros((2,2))
kappa_final = 0.0

for step in range(self.n_steps):
    psi = client.projection.lift(client.posterior_state.mean)
    psi = client.dynamics.step(psi, 0.04, rng=rng)
    psi_hat = -client.dynamics.dH_dpsi_conj(psi)
    z = client.projection.project(psi)
    client.posterior_state = FieldPosterior.full(z, client.posterior_state)

    # T_munu
    d = [np.gradient(psi.real, self.dx, axis=i) for i in range(2)]
    dh = [np.gradient(psi_hat.real, self.dx, axis=i) for i in range(2)]
    T_sp = np.zeros((2,2,N,N))
    for mu in range(2):
        for nu in range(2):
            T_sp[mu,nu] = dh[mu]*d[nu]+dh[nu]*d[mu]
    tr = sum(dh[i]*d[i] for i in range(2))
    T_sp[0,0] -= tr; T_sp[1,1] -= tr

    T_buffer.append(T_sp.mean(axis=(-2,-1)))
    if len(T_buffer) > self.T_window:
        T_buffer.pop(0)
    T_avg = np.mean(T_buffer, axis=0)

    # Poisson → h
    h_sp = np.zeros((2,2,N,N))
    for mu in range(2):
        for nu in range(2):

```

```

        Tk = np.fft.fft2(T_sp[mu,nu])
        hk = -16*math.pi*Tk/self.k2
        hk[0,0]=0.0
        h_sp[mu,nu] = np.fft.ifft2(hk).real
    h_avg = h_sp.mean(axis=(-2,-1))

    # G^(1) from h_sp (spatially averaged)
    # Simplified: for uniform h, G^(1) ~ 0; use k-space
    # G_hat(k) = (1/2)(...) - use direct formula for mean
    # At the mean level: G_avg ~ -h_avg * k_avg^2 (schematic)
    # Use proper computation
    def d2mean(f_sp, mu, nu):
        return np.gradient(
            np.gradient(f_sp[mu,nu], self.dx, axis=mu),
            self.dx, axis=nu
        ).mean()

    h_tr = h_sp[0,0]+h_sp[1,1]
    G_avg = np.zeros((2,2))
    divh = [sum(np.gradient(h_sp[mu,nu],self.dx,axis=mu)
                for mu in range(2)) for nu in range(2)]
    div2h = sum(d2mean(h_sp,mu,nu) for mu in range(2) for nu in range(2))
    lap_htr = ((np.roll(h_tr,1,0)+np.roll(h_tr,-1,0)+
                np.roll(h_tr,1,1)+np.roll(h_tr,-1,1)-4*h_tr)/self.dx**2).

    for mu in range(2):
        for nu in range(2):
            eta = 1.0 if mu==nu else 0.0
            t1 = np.gradient(divh[nu],self.dx,axis=mu).mean()
            t2 = np.gradient(divh[mu],self.dx,axis=nu).mean()
            lap_h = ((np.roll(h_sp[mu,nu],1,0)+np.roll(h_sp[mu,nu],-1,0)+
                    np.roll(h_sp[mu,nu],1,1)+np.roll(h_sp[mu,nu],-1,1)
                    -4*h_sp[mu,nu])/self.dx**2).mean()
            t3 = -lap_h
            t4 = -d2mean(np.array([[h_tr,h_tr],[h_tr,h_tr]]),mu,nu)
            t5 = -eta*(div2h - lap_htr)
            G_avg[mu,nu] = 0.5*(t1+t2+t3+t4+t5)

    T_n = np.linalg.norm(T_avg)+1e-15
    G_n = np.linalg.norm(G_avg)+1e-15
    kappa = float(np.sum(T_avg*G_avg)/G_n**2)
    res = float(np.linalg.norm(T_avg - kappa*G_avg)/T_n)
    residuals.append(res)

    rec = friction.update(client.posterior_state, step*0.04)
    if rec.closed and closure_step is None:
        closure_step = step

```

```

        z_final = z.copy()
        h_final = h_avg.copy()
        T_final = T_avg.copy()
        G_final = G_avg.copy()
        kappa_final = kappa

    if closure_step is None:
        z_final = client.posterior_state.mean.copy()
        h_final = h_avg
        T_final = T_avg
        G_final = G_avg
        kappa_final = kappa

    return TrajectoryResult(
        seed=seed,
        einstein_residual_final=residuals[-1] if residuals else 1.0,
        einstein_residual_min=min(residuals) if residuals else 1.0,
        closure_step=closure_step,
        z_final=z_final,
        h_final=h_final,
        kappa=kappa_final,
    )

def find(self, verbose: bool = True) -> GRFixedPoint:
    if verbose:
        print(f"  Running {self.n_traj} trajectories to find GR fixed point..")
        print(f"  {'seed':>5}  {'min_res':>10}  {'final_res':>11}  {'closure'

    results = []
    for seed in range(self.n_traj):
        r = self._run_trajectory(seed)
        results.append(r)
        if verbose:
            cl = str(r.closure_step) if r.closure_step else "none"
            print(f"  {seed:>5}  {r.einstein_residual_min:>10.6f}  "
                  f"{r.einstein_residual_final:>11.6f}  {cl:>8}")

    # Find seed with minimum Einstein residual
    best = min(results, key=lambda r: r.einstein_residual_min)

    if verbose:
        print(f"\n  Best seed: {best.seed}  "
              f"min_residual={best.einstein_residual_min:.6f}")

    # Rerun best seed with more steps for refinement
    if verbose:
        print(f"  Refining from seed {best.seed}...")

```

```
refined = self._run_trajectory(best.seed)

return GRFixedPoint(
    z_star=refined.z_final,
    h_star=refined.h_final,
    T_star=np.zeros((2,2)), # filled below
    G_star=np.zeros((2,2)),
    einstein_residual=refined.einstein_residual_min,
    kappa=refined.kappa,
    seed_used=best.seed,
    n_trajectories=self.n_traj,
)
```